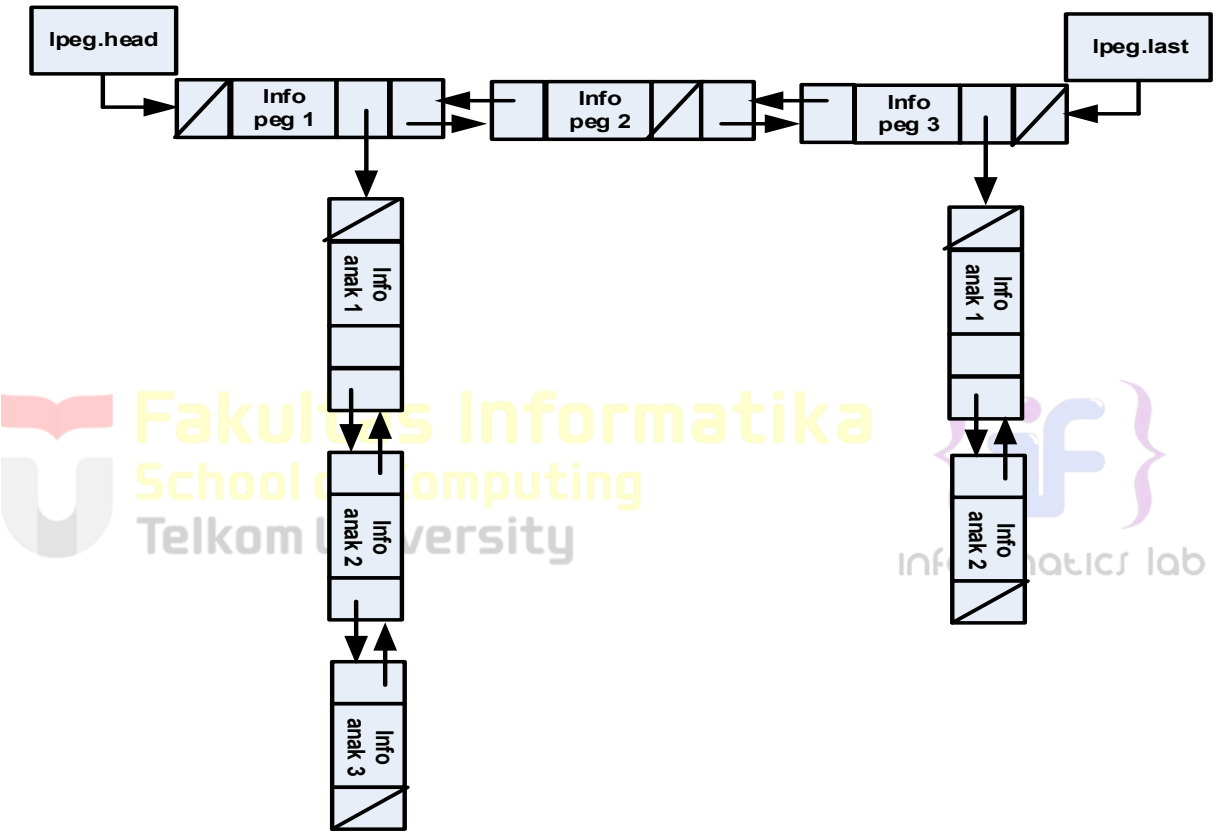
Modul 12 MULTI LINKED LIST

TUJUAN PRAKTIKUM
1. Memahami penggunaan *Multi Linked list*. 2. Mengimplementasikan *Multi Linked list* dalam beberapa studi kasus.

12.1 *Multi Linked List*

Multi List merupakan sekumpulan *list* yang berbeda yang memiliki suatu keterhubungan satu sama lain. Tiap elemen dalam *multi link list* dapat membentuk *list* sendiri.Biasanya ada yang bersifat sebagai *list* induk dan *list* anak .

Contoh *Multi Linked list* dapat dilihat pada gambar berikut.



Gambar 12-1 Multi *Linked list*

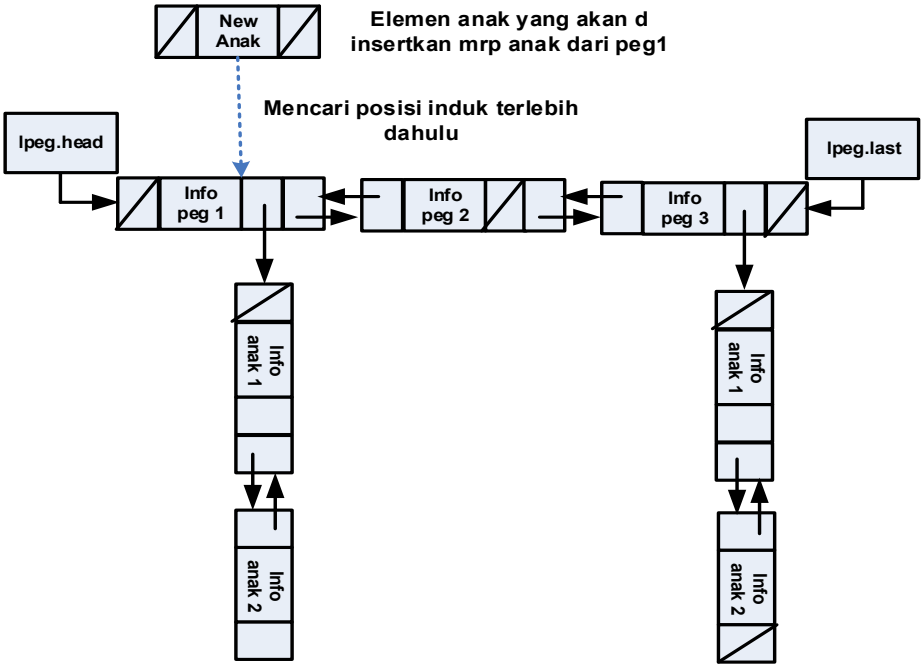
Jadi , dari implementasi di atas akan terdapat dua buah *list*, *list* pegawai dan *list* anak. Dimana untuk *list* pegawai menunjuk satu buah *list* anak. Disini *list* induknya adalah *list* pegawai dan *list* anaknya adalah *list* anak.

12.1.1 Insert

A. *Insert* Anak

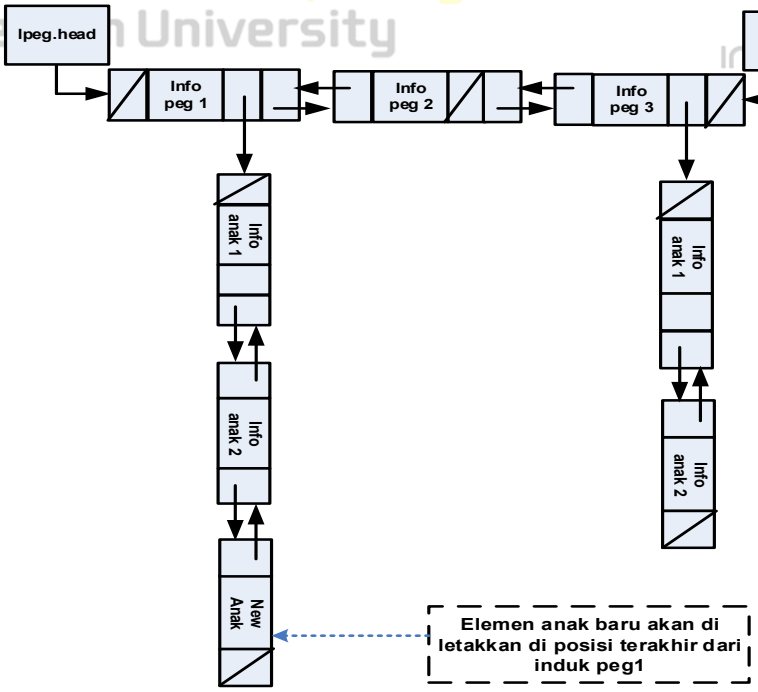
Dalam penambahan elemen anak harus diketahui dulu elemen induknya.

Berikut ini ilustrasi *insert* anak dengan konsep *insert last*:



Gambar 12-2 *Multi Linked list Insert* Anak 1

Fakultas Informatika
School of Computing
Telkom University
informatics lab



Gambar 12-3 *Multi Linked list Insert* Anak 2

```
/* buat dahulu elemen yang akan disisipkan */
address_anak alokasiAnak(infotypeanak X){
```

```
    address_anak P = alokasi(X);
    P→next = Nil;
    P→prev = Nil;
    return P;
}
/* mencari apakah ada elemen pegawai dengan info X */
address findElm(listinduk L, infotypeinduk X){
    address cariInduk = L.first;
    do{
        if(cariInduk.info == X){
            return cariInduk;
        }else{
            cariInduk = cariInduk→next;
        }
    }while(cariInduk.info != X || cariInduk != L→last)
}
/* menyisipkan anak pada akhir list anak */
void insertLastAnak(listanak &Lanak, address_anak P){
    address_anak ! = head(&Lanak);
    do{
        Q = Q→next;
    }while(next(&Lanak)!=Nil)
    Q→next = P;
    P→prev = Q;
    P→next = Nil;
}
```

B. *Insert* Induk

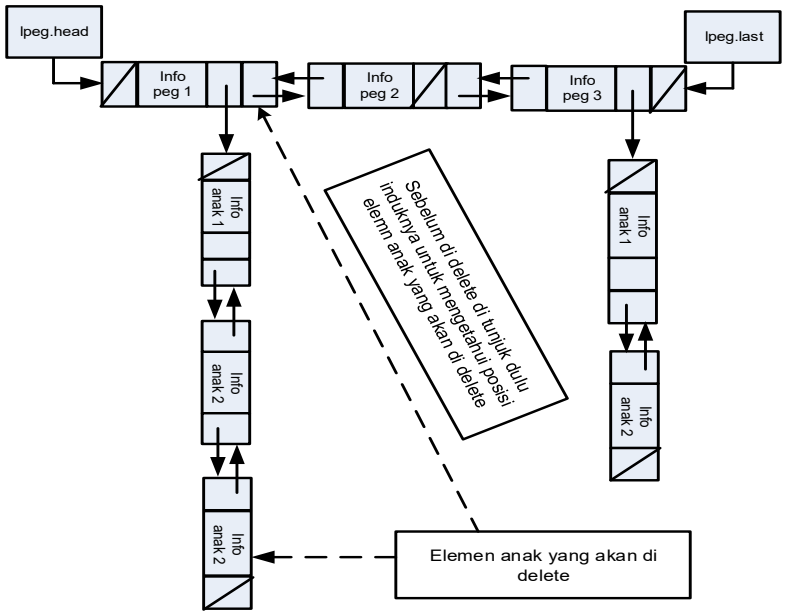
Untuk *insert* elemen induk sama dengan konsep *insert* pada *Singly*, *Doubly* dan *Circular Linked List*.

Fakultas Informatika
School of Computing
Telkom University
Informatics lab

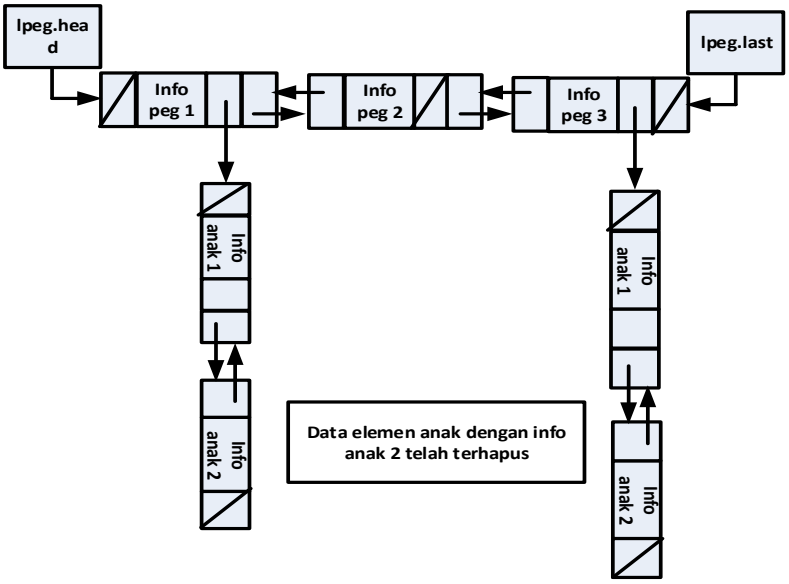
12.1.2 Delete

A. *Delete* Anak

Sama dengan *insert* anak untuk *delete* anak maka harus diketahui dulu induknya.Berikut ini Gambar ilustrasinya untuk *delete last* pada induk peg 1:



Gambar 12-4 *Multi Linked list Delete* Anak 1

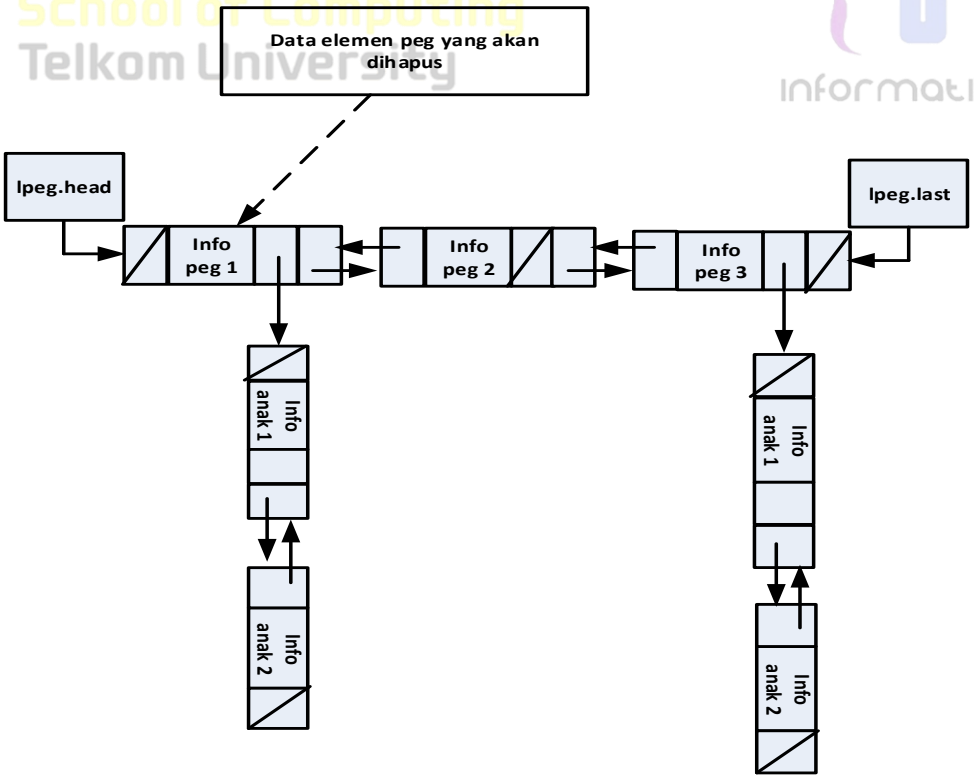


Gambar 12-5 *Multi Linked list Delete* Anak 2

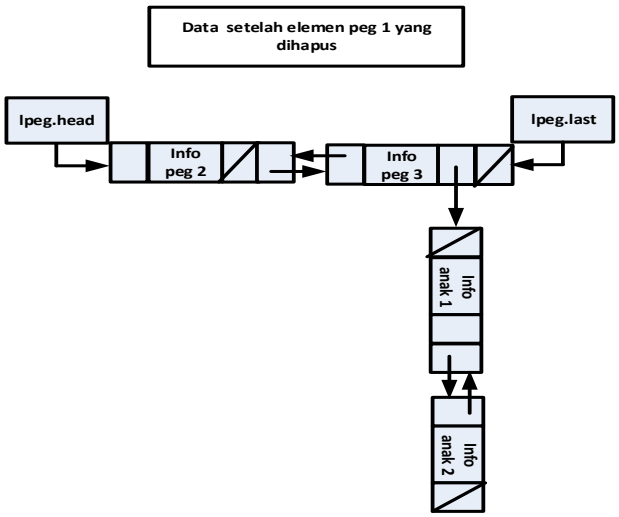
B. *Delete* Induk

Untuk *delete* elemen induk maka saat di hapus maka seluruh anak dengan induk tersebut juga harus dihapus. Berikut ini gambar ilustrasinya:

Fakultas Informatika
School of Computing
Telkom University
informatics lab



Gambar 12-6 *Multi Linked list Delete* Induk 1



Gambar 12-7 *Multi Linked list Delete* Induk 2

```
1   /*file : multilist .h*/
2   /* contoh ADT list berkait dengan representasi fisik pointer*/
3   /* representasi address    dengan pointer*/
4
5   /* info tipe adalah integer */
6   #ifndef MULTILIST_H_INCLUDED
7   #define MULTILIST_H_INCLUDED
8   #define Nil NULL
9
10  typedef int infotypeanak;
11  typedef int infotypeinduk;
12  typedef struct elemen_list_induk *address;
13  typedef struct elemen_list_anak *address_anak;
14  /* define list : */
15
16  /* list kosong jika L.first=Nil
17  setiap elemen address P dapat diacu P→info atau P→next
18  elemen terakhir list jika addressnya last, maka L.last = Nil */
19  struct elemen_list_anak{
20  /* struct ini untuk menyimpan elemen anak dan pointer penunjuk
21     elemen tetangganya */
22      infotypeanak info;
23      address_anak next;
24      address_anak prev;
25  };
26
27  struct listanak {
28  /* struct ini digunakan untuk menyimpan list anak itu sendiri */
29      address_anak first;
30      address_anak last;
31  };
32
33  struct elemen_list_induk{
34  /* struct ini untuk menyimpan elemen induk dan pointer penunjuk
35     elemen tetangganya */
36      infotypeanak info;
        listanak lanak;
37      address next;
38      address prev;
39  };
40  struct listinduk {
41  /* struct ini digunakan untuk menyimpan list induk itu sendiri */
42      address first;
43      address last;
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

```
};

/********* pengecekan apakah list kosong ***********/
boolean ListEmpty(listinduk L);
/*mengembalikan nilai true jika list induk kosong*/
boolean ListEmptyAnak(listanak L);
/*mengembalikan nilai true jika list anak kosong*/

/********* pembuatan list kosong *********/
void CreateList(listinduk &L);
/* I.S. sembarang
   F.S. terbentuk list induk kosong*/
void CreateListAnak(listanak &L);
/* I.S. sembarang
   F.S. terbentuk list anak kosong*/

/********* manajemen memori *********/
address alokasi(infotypeinduk P);
/* mengirimkan address dari alokasi sebuah elemen induk
   jika alokasi berhasil, maka nilai address tidak Nil dan jika gagal
   nilai address Nil */

address_anak alokasiAnak(infotypeanak P);
/* mengirimkan address dari alokasi sebuah elemen anak
   jika alokasi berhasil, maka nilai address tidak Nil dan jika gagal
   nilai address_anak Nil */

void dealokasi(address P);
/* I.S. P terdefinisi
   F.S. memori yang digunakan P dikembalikan ke sistem */

void dealokasiAnak(address_anak P);
/* I.S. P terdefinisi
   F.S. memori yang digunakan P dikembalikan ke sistem */
/********* pencarian sebuah elemen list *********/
address findElm(listinduk L, infotypeinduk X);
/* mencari apakah ada elemen list dengan P→info = X
   jika ada, mengembalikan address elemen tab tsb, dan Nil jika sebaliknya
*/
address_anak findElm(listanak Lanak, infotypeanak X);
/* mencari apakah ada elemen list dengan P→info = X
   jika ada, mengembalikan address elemen tab tsb, dan Nil jika sebaliknya
*/
boolean fFindElm(listinduk L, address P);
/* mencari apakah ada elemen list dengan alamat P
   mengembalikan true jika ada dan false jika tidak ada */
boolean fFindElmanak(listanak Lanak, address_anak P);
/* mencari apakah ada elemen list dengan alamat P
   mengembalikan true jika ada dan false jika tidak ada */

address findBefore(listinduk L, address P);
/* mengembalikan address elemen sebelum P
   jika P berada pada awal list, maka mengembalikan nilai Nil */
address_anak findBeforeAnak(listanak Lanak, infotypeinduk X, address_anak
P);
/* mengembalikan address elemen sebelum P dimana P→info = X
   jika P berada pada awal list, maka mengembalikan nilai Nil */

/********* penambahan elemen **********/
void insertFirst(listinduk &L, address P);
/* I.S. sembarang, P sudah dialokasikan
   F.S. menempatkan elemen beralamat P pada awal list */

void insertAfter(listinduk &L, address P, address Prec);
/* I.S. sembarang, P dan Prec alamt salah satu elemen list
   F.S. menempatkan elemen beralamat P sesudah elemen beralamat Prec */

```

```
111 void insertLast(listinduk &L, address P);
112 /* I.S. sembarang, P sudah dialokasikan
113    F.S. menempatkan elemen beralamat P pada akhir list */
114
115 void insertFirstAnak(listanak &L, address_anak P);
116 /* I.S. sembarang, P sudah dialokasikan
117    F.S. menempatkan elemen beralamat P pada awal list */
118
119 void insertAfterAnak(listanak &L, address_anak P, address_anak Prec);
120 /* I.S. sembarang, P dan Prec alamt salah satu elemen list
121    F.S. menempatkan elemen beralamat P sesudah elemen beralamat Prec */
122
123 void insertLastAnak(listanak &L, address_anak P);
124 /* I.S. sembarang, P sudah dialokasikan
125    F.S. menempatkan elemen beralamat P pada akhir list */
126
127 /********* penghapusan sebuah elemen *********/
128 void delFirst(listinduk &L, address &P);
129 /* I.S. list tidak kosong
130    F.S. adalah alamat dari alamat elemen pertama list
131    sebelum elemen pertama list dihapus
132    elemen pertama list hilang dan list mungkin menjadi kosong
133    first elemen yang baru adalah successor first elemen yang lama */
134 void delLast(listinduk &L, address &P);
135 /* I.S. list tidak kosong
136    F.S. adalah alamat dari alamat elemen terakhir list
137    sebelum elemen terakhir list dihapus
138    elemen terakhir list hilang dan list mungkin menjadi kosong
139    last elemen yang baru adalah successor last elemen yang lama */
140
141 void delAfter(listinduk &L, address &P, address Prec);
142 /* I.S. list tidak kosng, Prec alamat salah satu elemen list
143    F.S. P adalah alamatdari Prec→next, menghapus Prec→next dari list */
144 void delP (listinduk &L, infotypeinduk X);
145 /* I.S. sembarang
146    F.S. jika ada elemen list dengan alamat P, dimana P→info = X,
147         maka P dihapus dan P di-dealokasi,
148         jika tidak ada maka list tetap list mungkin akan menjadi kosong
149         karena penghapusan */
150
151 void delFirstAnak(listanak &L, address_anak &P);
152 /* I.S. list tidak kosong
153    F.S. adalah alamat dari alamat elemen pertama list
154    sebelum elemen pertama list dihapus
155    elemen pertama list hilang dan list mungkin menjadi kosong
156    first elemen yang baru adalah successor first elemen yang lama */
157 void delLastAnak(listanak &L, address_anak &P);
158 /* I.S. list tidak kosong
159    F.S. adalah alamat dari alamat elemen terakhir list
160    sebelum elemen terakhir list dihapus
161    elemen terakhir list hilang dan list mungkin menjadi kosong
162    last elemen yang baru adalah successor last elemen yang lama */
163
164 void delAfterAnak(listanak &L, address_anak &P, address_anak Prec);
165 /* I.S. list tidak kosng, Prec alamat salah satu elemen list
166    F.S. P adalah alamatdari Prec→next, menghapus Prec→next dari list */
167 void delPAnak (listanak &L, infotypeanak X);
168 /* I.S. sembarang
169    F.S. jika ada elemen list dengan alamat P, dimana P→info = X,
170         maka P dihapus dan P di-dealokasi,
171         jika tidak ada maka list tetap list mungkin akan menjadi kosong
172         karena penghapusan */
173 /********** proses semau elemen list *********/
174 void printInfo(list L);
175 /* I.S. list mungkin kosong
176    F.S. jika list tidak kosong menampilkan semua info yang ada pada list
177 */
```

```
178
179   int nbList(list L);
180   /* mengembalikan jumlah elemen pada list */
181
182   void printInfoAnak(listanak Lanak);
183   /* I.S. list mungkin kosong
184      F.S. jika list tidak kosong menampilkan semua info yang ada pada list
185   */
186
187   int nbListAnak(listanak Lanak);
188   /* mengembalikan jumlah elemen pada list anak */
189
190   /********** proses terhadap list **********/
191   void delAll(listinduk &L);
192   /* menghapus semua elemen list dan semua elemen di-dealokasi */
193
194   #endif
195
196
197
198
199
```

12.2 *Latihan*

2. Perhatikan program 46 **multilist.h**, buat **multilist.cpp** untuk implementasi semua fungsi pada **multilist.h**. Buat **main.cpp** untuk pemanggilan fungsi-fungsi tersebut.

3. Buatlah ADT *Multi Linked list* sebagai berikut di dalam *file* **"circularlist.h"**:

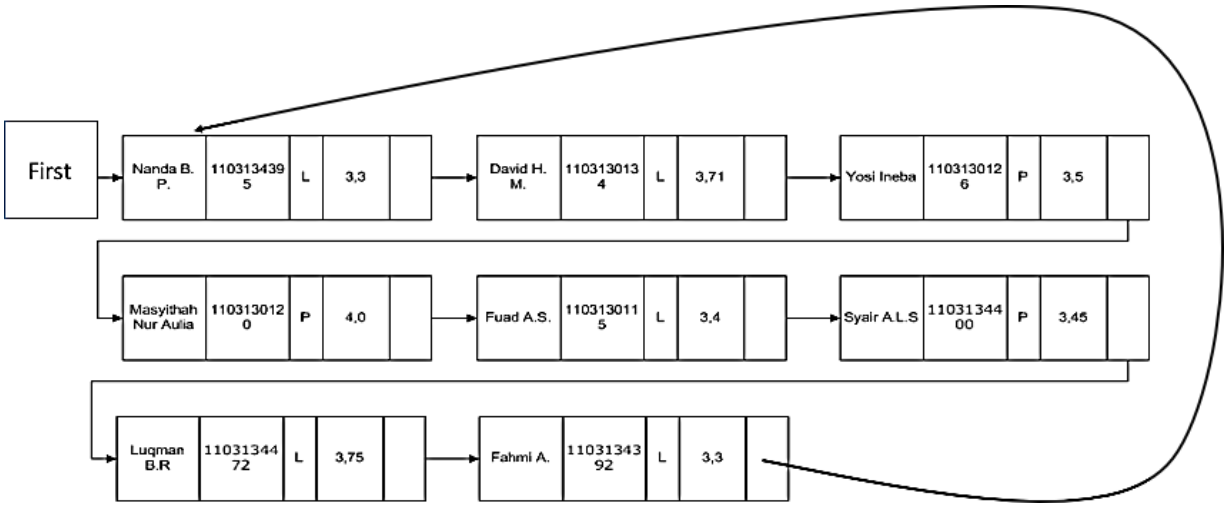
```
Type infotype : mahasiswa <
    Nama:string
    Nim:string
    Jenis_kelamin:char
    Ipk:float>
Type address : pointer to ElmList
Type ElmList <
    info : infotype
    next :address>
Type List <
    First : address>
```

- Terdapat 11 fungsi/prosedur untuk ADT circularlist
  - procedure **CreateList**(input/output L : List)
  - function **alokasi**(x : infotype) → address
  - procedure **dealokasi**(input/output t P : address)
  - procedure **insertFirst**(input/output L : List, input P : address)
  - procedure **insertAfter**(input/output L : List, input Prec : address, P : address)
  - procedure **insertLast**(input/output L : List, input P : address)
  - procedure **deleteFirst**(input/output L : List, input/output P : address)
  - procedure **deleteAfter**(input/output L : List, input Prec : address, input/output t P : address)
  - procedure **deleteLast**(input/output L : List, P : address)
  - function **findElm**(L : List, x : infotype) → address
  - procedure **printInfo**(input L : List)

Keterangan :

- fungsi findElm mencari elemen di dalam *list* L berdasarkan nim
  - fungsi mengembalikan elemen dengan dengan info nim == x.nim jika ditemukan

- fungsi mengembalikan NIL jika tidak ditemukan



Gambar 12-8 Ilustrasi data

Buatlah implementasi ADT *Doubly Linked list* pada *file* "**circularlist.cpp**"**.** Tambahkan fungsi/prosedur berikut pada *file* **"main.cpp".**

- fungsi **create (in nama, nim : string, jenis_kelamin : char, ipk : float)**
  - **fungsi disediakan**, ketik ulang code yang diberikan
  - fungsi mengalokasikan sebuah elemen *list* dengan info sesuai *input*

```
address createData(string nama, string nim, char jenis_kelamin, float ipk)
{
    /**
    * PR : mengalokasikan sebuah elemen list dengan info dengan info sesuai input
    * FS : address P menunjuk elemen dengan info sesuai input
    */
    infotype x;
    address P;
    x.nama = nama;
    x.nim = nim;
    x.jenis_kelamin = jenis_kelamin;
    x.ipk = ipk;
    P = alokasi(x);
    return P;
}
```

Gambar 12-9 Fungsi *create*

Cobalah hasil implementasi ADT pada *file* "**main.cpp**"

```
int main()
{
    List L, A, B, L2;
    address P1 = Nil;
    address P2 = Nil;
    infotype x;
    createList(L);

    cout<<"coba insert first, last, dan after"<<endl;
    P1 = createData("Danu", "04", 'l', 4.0);
    insertFirst(L,P1);

    P1 = createData("Fahmi", "06", 'l',3.45);
    insertLast(L,P1);
    P1 = createData("Bobi", "02", 'l',3.71);
```

```
    insertFirst(L,P1);

    P1 = createData("Ali", "01", 'l', 3.3);
    insertFirst(L,P1);

    P1 = createData("Gita", "07", 'p', 3.75);
    insertLast(L,P1);

    x.nim = "07";
    P1 = findElm(L,x);
    P2 = createData("Cindi", "03", 'p', 3.5);
    insertAfter(L, P1, P2);

    x.nim = "02";
    P1 = findElm(L,x);
    P2 = createData("Hilmi", "08", 'p', 3.3);
    insertAfter(L, P1, P2);

    x.nim = "04";
    P1 = findElm(L,x);
    P2 = createData("Eli", "05", 'p', 3.4);
    insertAfter(L, P1, P2);
    printInfo(L);
    return 0;
}
```

Gambar 12-10 main.cpp

```
coba insert first, last, dan after
Nama : Ali
NIM  : 01
L/P  : l
IPK  : 3.3

Nama : Bobi
NIM  : 02
L/P  : l
IPK  : 3.71

Nama : Cindi
NIM  : 03
L/P  : p
IPK  : 3.5

Nama : Danu
NIM  : 04
L/P  : l
IPK  : 4

Nama : Eli
NIM  : 05
L/P  : p
IPK  : 3.4

Nama : Fahmi
NIM  : 06
L/P  : l
IPK  : 3.45

Nama : Gita
NIM  : 07
L/P  : p
IPK  : 3.75

Nama : Hilmi
NIM  : 08
L/P  : l
IPK  : 3.3
```

Gambar 12-11 *Output*

Fakultas Informatika
School of Computing
Telkom University
informatics lab